

Intermediate Language (ILDASM & ILASM) in C#.NET

In this article, I will discuss the **Intermediate Language (ILDASM & ILASM) Code in C#** with Examples. Please read our previous article, where we discussed the [.NET Program Execution Process Flow](#) in detail. **ILDASM** stands for Intermediate Language Disassembler and **ILASM** stands for Intermediate Language Assembler. As part of this article, we will discuss the following pointers, and at the end of this article, you will understand all about Intermediate Language (IL Code) in C#.

1. [What happens when we compile a .NET Application?](#)
2. [Understanding the Intermediate Language \(IL Code\) in C#?](#)
3. [What are ILDASM and ILASM?](#)
4. [How do you view the Intermediate Language code in C#?](#)
5. [What is Manifest?](#)
6. [How do you export the Intermediate Language code to a text file?](#)
7. [How do you rebuild an assembly from a text file that contains manifest and IL?](#)

What happens when we compile a .NET Application?

When we compile any .NET application, it will generate an assembly with the extension of either a .DLL or an .EXE. For example, if you compile a Windows or Console application, then you will get an .EXE, whereas if you compile a Web or Class library project, then you will get a .DLL. Irrespective of whether it is a .DLL or .EXE, an assembly consists of two things i.e. **Manifest and Intermediate language**. Let us understand what the Intermediate Language and Manifest look like in .NET Framework with an example.

Understanding Intermediate Language (ILDASM and ILASM) Code in C#:

In order to understand Intermediate Language Code (ILDASM and ILASM) in C#, let us create a simple console application. Once you create the console application, please modify the Program class as shown below.

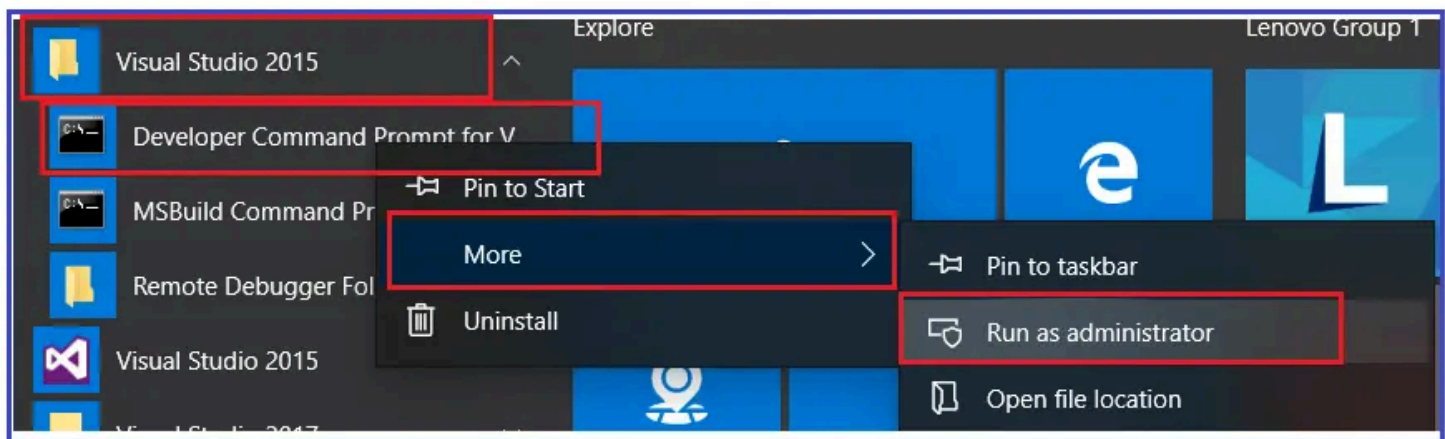
```
using System;
namespace ILDASMDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Understanding ILDASM and ILASM");
            Console.Read();
        }
    }
}
```

Now, build the application. Once you build the application, the above source code is compiled and intermediate language code is generated and packaged into an assembly. In order to see the assembly, just right-click on the Project and select **Open Folder in the File Explorer** option and then go to the **bin => Debug** folder and you should see an assembly with .exe extension generated as shown in the below image as it is a console application.

This PC > LENOVO (D:) > ILDASMDemo > ILDASMDemo > bin > Debug			
Name	Date modified	Type	Size
ILDASMDemo	23-02-2020 21:39	Application	5 KB
ILDASMDemo.exe	23-02-2020 21:39	XML Configuration...	1 KB
ILDASMDemo.pdb	23-02-2020 21:39	Program Debug D...	12 KB
ILDASMDemo.vshost	23-02-2020 21:39	Application	23 KB
ILDASMDemo.vshost.exe	23-02-2020 21:39	XML Configuration...	1 KB
ILDASMDemo.vshost.exe.manifest	19-03-2019 10:16	MANIFEST File	1 KB

How to view the Intermediate Language Code in .NET Framework?

The .NET framework provides a nice tool called **ILDASM (Intermediate Language DisAssembler)** to view the code of the intermediate language in .NET Framework. In order to use the ILDASM tool, you need to follow the below steps. Open Visual Studio Command Prompt in Administrator mode as shown in the below image.

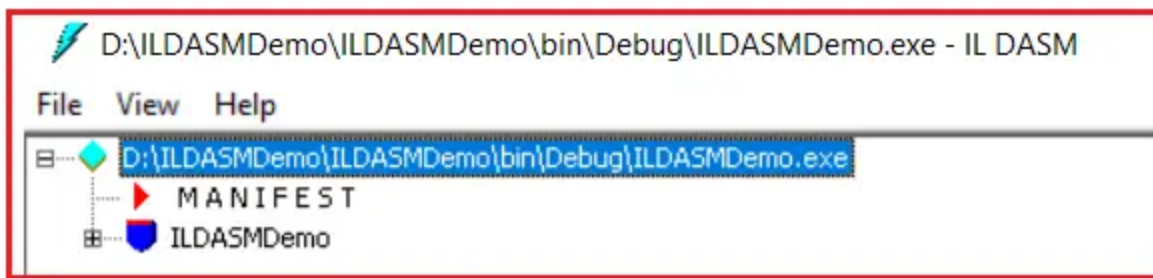


Once you open the Visual Studio command prompt in administrative mode, then type the “**ildasm.exe C:\YourDirectoryPath\YourAssembly.exe**” command and press enter. Here, you need to provide the exe path where your exe is generated. My EXE is generated in the path “**D:\ILDASMDemo\ILDASMDemo\bin\Debug\ILDASMDemo.exe**”, so I execute the following code in the command prompt:

```
Administrator: Developer Command Prompt for VS2015

C:\WINDOWS\system32>ildasm.exe D:\ILDASMDemo\ILDASMDemo\bin\Debug\ILDASMDemo.exe
```

Once you type the above command and press enter, it should open the following ILDASM window.



As you can see, the assembly consists of two things (**Manifest and Intermediate language**). Let us first discuss the intermediate language code, and then we will discuss what Manifest is. Now, let us expand the ILDASMDemo and compare it with our code. For a better understanding, please have a look at the below image. There is a constructor in ILDASM, and this is because, by default, the .NET Framework provides a default constructor when there is no constructor in your class. You can also see the Main method in the intermediate language code/

```

namespace ILDASMDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Understanding ILDASM and ILASM");
            Console.Read();
        }
    }
}

```

Namespace → ILDASMDemo

Class name → ILDASMDemo.Program

Constructor → .ctor : void()

Main Method → Main : void(string[])

Now, double-click on the Main method on the ILDASM window to see the intermediate language generated for the Main method, as shown below.

```

namespace ILDASMDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Understanding ILDASM and ILASM");
            Console.Read();
        }
    }
}

```

High Level Language C#

Intermediate Language

DoubleClick here to see the IL Code

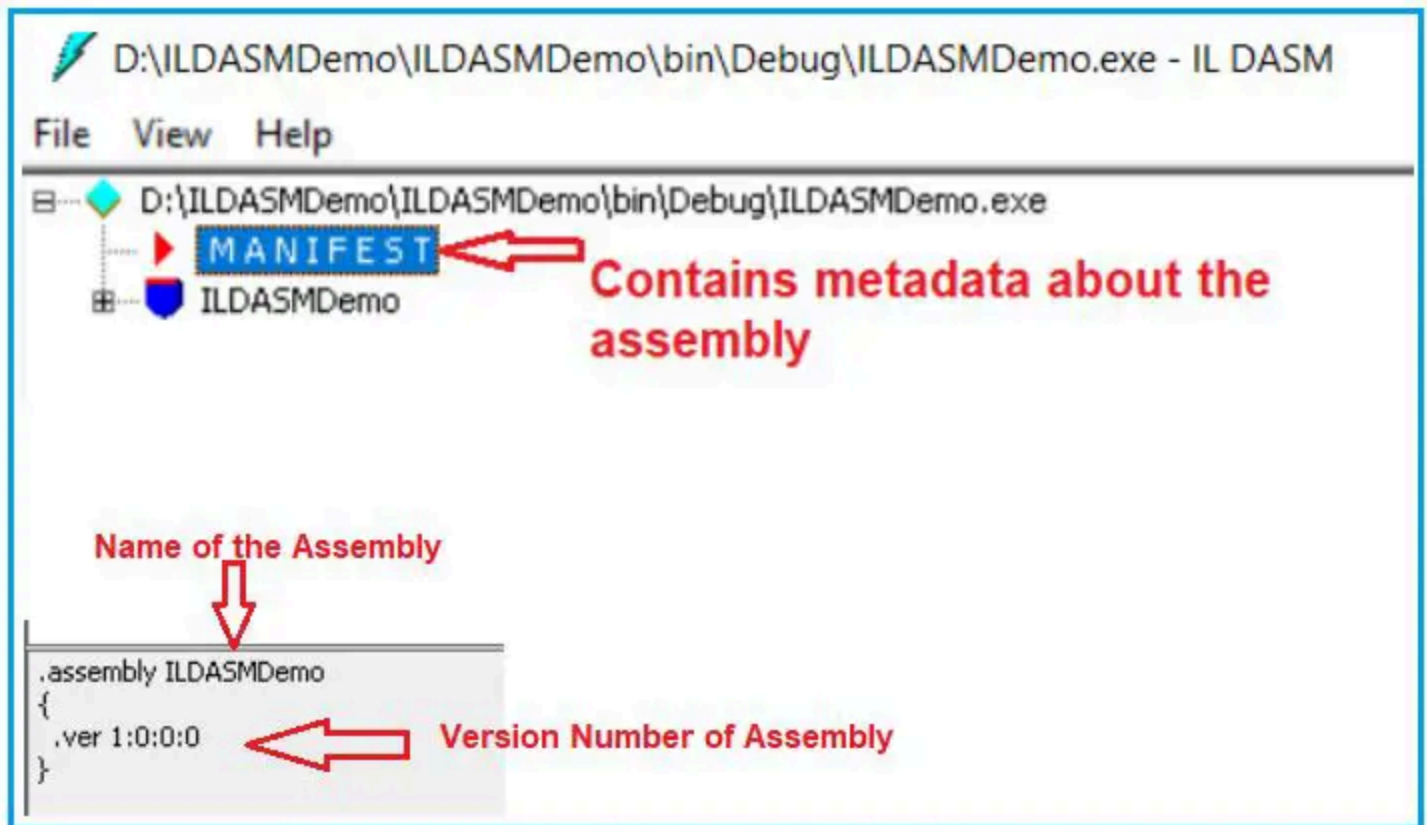
```

ILDASMDemo.Program::Main : void(string[])
.method private hidebysig static void Main(string[] args) cil managed
{
    .entrypoint
    // Code size          19 (0x13)
    .maxstack 8
    IL_0000: nop
    IL_0001: ldstr          "Understanding ILDASM and ILASM"
    IL_0006: call           void [mscorlib]System.Console::WriteLine(string)
    IL_000b: nop
    IL_000c: call           int32 [mscorlib]System.Console::Read()
    IL_0011: pop
    IL_0012: ret
} // end of method Program::Main

```

What is Manifest?

The Manifest contains metadata about the assembly, like the name of the assembly, the version number of the assembly, culture, and strong name information, as shown in the below image.



Metadata also contains information about the referenced assemblies. Each reference includes the dependent assembly's name, assembly metadata (version, culture, operating system, and so on), and public key if the assembly is strongly named.

How do you change Assembly info?

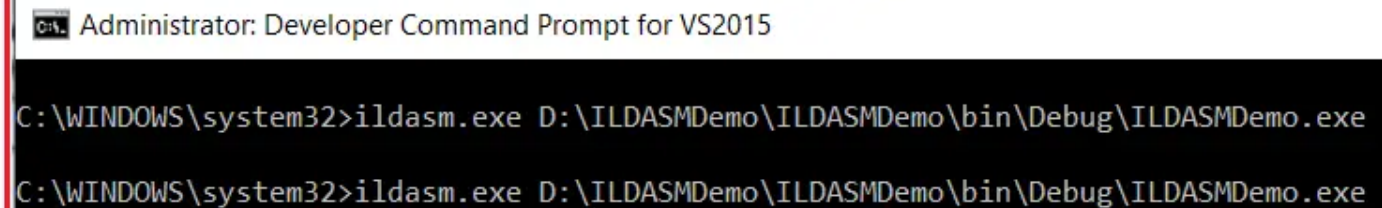
It is also possible to change or modify some of the information in the assembly manifest using attributes. For example, if you want to modify the version number, then you need to follow the below steps. Open the **AssemblyInfo.cs** class file, which is present under the Properties folder, as shown below. Every project in .NET has a properties folder.



In this file, you will find one attribute called the `AssemblyVersion` attribute, which is, by default, set to 1.0.0.0. Now, change this value to 2.0.0.0 as shown below.

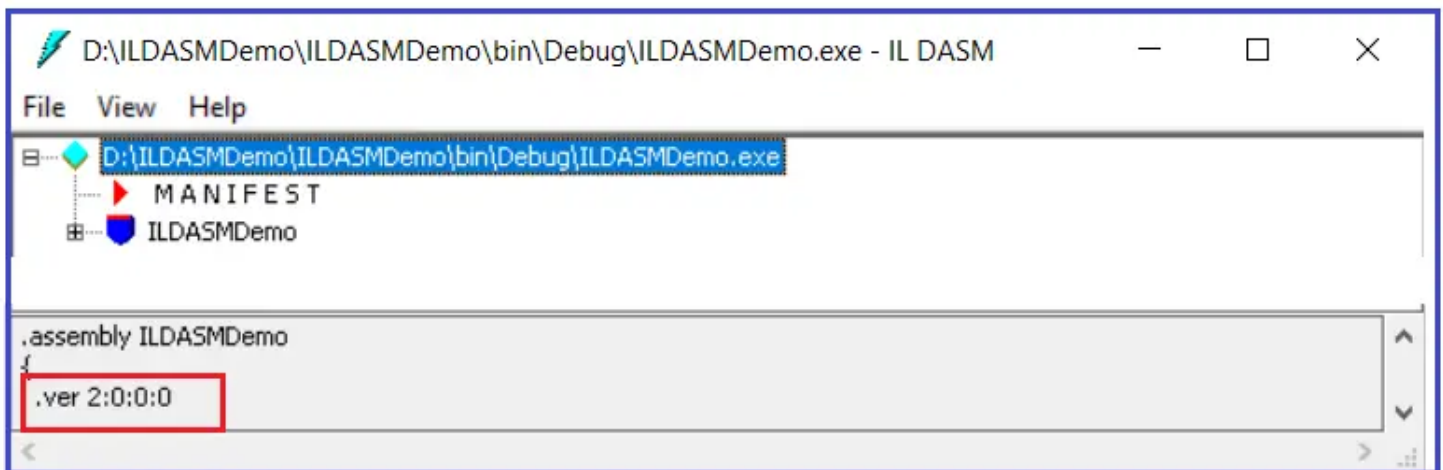
```
// Version information for an assembly consists of the following four values:  
//  
//     Major Version  
//     Minor Version  
//     Build Number  
//     Revision  
//  
// You can specify all the values or you can default the Build and Revision Numbers  
// by using the '*' as shown below:  
// [assembly: AssemblyVersion("1.0.*")]  
[assembly: AssemblyVersion("2.0.0.0")]  
[assembly: AssemblyFileVersion("1.0.0.0")]
```

Now, rebuild the solution. But before that, close the ILDASM window; otherwise, you will get an error. Once you rebuild the solution, open the assembly using the same **ILDASM.exe** in the command prompt as shown below.



```
C:\WINDOWS\system32>ildasm.exe D:\ILDASMDemo\ILDASMDemo\bin\Debug\ILDASMDemo.exe  
C:\WINDOWS\system32>ildasm.exe D:\ILDASMDemo\ILDASMDemo\bin\Debug\ILDASMDemo.exe
```

Once you execute the above command, it should open the assembly. At the bottom, you can find the updated version number of the assembly as expected as shown below.



How to export the Intermediate Language code to a text file?

If you want to export or save the Intermediate Language code into a text file, then you need to follow the below steps.

Select **File Menu** Option from the **ILDASM tool** and then select **Dump** and you will see "**Dump Options Window**", click on the **OK** button on the "Dump Options Window" as shown below.

Dump options

Encoding

☐ ANSI ☒ UTF-8 ☐ Unicode

☐ Dump Class List

☐ Dump Statistics

☒ Show Progress Bar

☐ Dump Header

☒ Dump IL Code

☐ Token Values

☐ Actual Bytes

☐ Line Numbers

☐ Source Lines

☒ Expand try/catch

☐ Dump Metainfo

☐ More HEX

☐ Raw: Counts, Sizes

☐ Raw: Header

☐ Raw: Header, Schema

☐ Raw: Header, Schema, Rows

☐ Raw: Heaps

☐ Unresolved Externals

☐ Validate

OK Cancel

Now you need to enter the file name of your choice. I am entering the file name as **MyFile** and saving it to the **D:** drive. Now navigate to D: drive in windows explorer and you should see MyFile.il Now, open MyFile.il with Notepad and you should see assembly metadata and IL code.

How to rebuild an assembly from a text file which contains manifest and IL?

If you want to rebuild an assembly from IL code then you need to use a tool called ILASM.exe. So, let's go and create an assembly from the file (MyFile.il) that we just save. In order to rebuild an assembly, please follow the below steps.

Type the following command in "Visual Studio Command Prompt" and press enter

ILASM.exe D:\MyFile.il

Now navigate to D: drive in windows explorer and you should see MyFile.exe. So, in short, we use **ILASM.exe** (Intermediate Language Assembler) to reconstruct an assembly from a text file that contains manifest and IL.

In the next article, I am going to discuss **CTS (Common Type System)** in detail. Here, in this article, I try to explain **Intermediate Language Code in C#**. I hope you understand ILDASM and ILASM tools and enjoy this IL Code in the C#.NET article.